

S11L5 - Bonus 2

Attacco a un database MySQL (SQL Injection)

PARTE 1 - APERTURA DEL FILE PCAP

Il file è stato aperto sulla VM CyberOps Workstation, avviando Wireshark (Applicazioni > CyberOPS > Wireshark) e caricando il file SQL_Lab.pcap situato nella directory /home/analyst/lab.support.files/.

La cattura si estende per un periodo di circa 8 minuti (441 secondi), durata complessiva dell'attacco di SQL injection osservato.

Domanda: Quali sono i due indirizzi IP coinvolti in questo attacco di SQL injection?

Risposta: Gli indirizzi IP coinvolti sono 10.0.2.4 (l'aggressore, sorgente del traffico) e 10.0.2.15 (la vittima, il server che ospita l'applicazione web DVWA e il database MySQL).

PARTE 2 - VISUALIZZARE L'INIZIO DELL'ATTACCO

Seguendo il flusso HTTP del pacchetto n. 13 e cercando la stringa "1=1", si osserva che l'aggressore ha inserito la query 1=1 nel campo di ricerca UserID dell'applicazione. Invece di restituire un messaggio di login fallito, l'applicazione ha risposto con un record proveniente dal database. Questo ha confermato all'aggressore che il campo era vulnerabile a SQL injection: la stringa 1=1 crea una condizione SQL sempre vera, quindi la query restituisce risultati indipendentemente dal valore realmente atteso.

PARTE 3 - L'ATTACCO CONTINUA

Seguendo il pacchetto n. 19 con la query 1' or 1=1 union select database(), user()#, l'aggressore ha ottenuto informazioni dirette sul database:

- Nome del database: dvwa
- Utente del database: root@localhost

Nella risposta compaiono inoltre più account utente dell'applicazione (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith, dvwa).

PARTE 4 - INFORMAZIONI DI SISTEMA

Seguendo il pacchetto n. 22 con la query 1' or 1=1 union select null, version()#, l'aggressore ha ottenuto la versione esatta del motore di database in esecuzione sul server.

Domanda: Qual è la versione?

Risposta: MySQL 5.7.12-0ubuntu1.1

PARTE 5 - INFORMAZIONI SULLE TABELLE

Seguendo il pacchetto n. 25 con la query `1' or 1=1 union select null, table_name from information_schema.tables#`, l'aggressore ha ottenuto l'elenco completo di tutte le tabelle presenti nel database, incluse tabelle di sistema (INNODB_SYS_COLUMNS, INNODB_SYS_TABLESTATS, columns_priv, db, engine_cost, event, ecc.) e, soprattutto, la tabella "users", quella potenzialmente contenente le credenziali degli account applicativi.

Domanda: Cosa farebbe per l'aggressore il comando modificato di `(1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users')`?

Risposta: Il database risponderebbe con un output molto più breve e mirato rispetto a quello ottenuto interrogando `information_schema.tables`. Invece di restituire i nomi di tutte le tabelle del database, questa query interroga la tabella di sistema `INFORMATION_SCHEMA.columns` filtrando specificamente per `table_name='users'`, restituendo quindi solo i nomi delle colonne della tabella users (ad esempio `user_id`, `first_name`, `last_name`, `user`, `password`, ecc.). Questo permette all'aggressore di sapere esattamente quali campi interrogare nel passo successivo dell'attacco, senza dover indovinare la struttura della tabella.

PARTE 6 - L'ATTACCO SI CONCLUDE

Seguendo il pacchetto n. 28 con la query `1' or 1=1 union select user, password from users#`, l'aggressore ha estratto direttamente dalla tabella users gli username e i relativi hash delle password di tutti gli account dell'applicazione.

Domanda: Quale utente ha l'hash della password di `8d3533d75ae2c3966d7e0d4fcc69216b`?

Risposta: L'utente è "1337".

Utilizzando il servizio online di cracking hash <https://crackstation.net/>, l'hash è stato decifrato con successo.

Domanda: Qual è la password in chiaro?

Risposta: La password in chiaro è "charley".

Questo rappresenta l'obiettivo finale dell'attacco: l'aggressore ora dispone di una coppia username/password valida (1337 / charley), che potrebbe utilizzare per accedere legittimamente al sistema qualora la password non venga cambiata o venga riutilizzata su altri servizi..

DOMANDE DI RIFLESSIONE

1. Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

I siti web sono comunemente basati su database relazionali e utilizzano il linguaggio SQL per interrogarli. Quando un'applicazione costruisce le query SQL concatenando direttamente l'input fornito dall'utente (come nel campo UserID di DVWA), diventa vulnerabile a SQL injection: un aggressore può alterare la logica della query originale per estrarre, modificare o cancellare dati a cui non dovrebbe avere accesso.

2. Due metodi per prevenire gli attacchi di SQL injection:

a) Utilizzo di query parametrizzate (prepared statements): invece di costruire le query SQL concatenando direttamente le stringhe inserite dall'utente, l'applicazione dovrebbe utilizzare istruzioni precompilate in cui i valori di input vengono passati come parametri separati dalla struttura della query. In questo modo il database interpreta sempre l'input come dato e mai come codice SQL eseguibile, neutralizzando alla radice tentativi come "1' or 1=1".

b) Validazione e filtraggio rigoroso dell'input dell'utente (input sanitization) e adozione di un Web Application Firewall (WAF): l'applicazione dovrebbe controllare che i dati inseriti rispettino il formato atteso (ad esempio un numero, un'email, una stringa senza caratteri speciali come apici o punti e virgola), rifiutando o neutralizzando input sospetti prima che raggiungano il database. Un WAF posto davanti all'applicazione può inoltre riconoscere e bloccare automaticamente pattern tipici di SQL injection nel traffico in transito, offrendo un ulteriore livello di difesa.

Altri accorgimenti utili includono: applicare il principio del privilegio minimo agli account del database (l'account usato dall'applicazione non dovrebbe avere permessi di amministratore), disabilitare funzionalità del database non necessarie, e monitorare/loggare le istruzioni SQL eseguite per individuare tempestivamente tentativi di attacco.